

Citizen Grievance & Sentiment Analysis System

SUBMITTED BY UVESH SHAIKH

Week 3 Report: Sentiment Analysis and Priority Scoring

1. Introduction

In Week 3, the focus of the project was to analyze the emotional tone of citizen complaints and assign priority levels based on their severity. Sentiment analysis was performed to classify complaints into different categories, enabling better understanding and prioritization of public issues.

2. Sentiment Analysis Approach

Since the dataset does not contain predefined sentiment labels, a lexicon-based approach was used for sentiment analysis. The TextBlob library was applied to determine the polarity of each complaint based on its textual content.

The complaints were classified into three categories:

- Positive
- Neutral
- Negative

```
•[27]: from textblob import TextBlob

#create Sentiment Function
##Sentiment analysis was performed using TextBlob to classify complaints into Positive, Neutral, and Negative categories.
def get_sentiment(text):
    polarity = TextBlob(text).sentiment.polarity

    if polarity > 0:
        return "Positive"
    elif polarity == 0:
        return "Neutral"
    else:
        return "Negative"
```

3. Applying Sentiment Analysis

The sentiment classification function was applied to the cleaned text data to generate a new column representing the sentiment of each complaint.

```
[28]: df['sentiment'] = df['clean_text'].apply(get_sentiment)
```

```
[29]: df[['clean_text', 'sentiment']].head()
```

```
[29]:
```

	clean_text	sentiment
0	noise commercial loud musicparty	Positive
1	noise residential bangingpounding	Neutral
2	illegal parking blocked hydrant	Negative
3	noise residential loud musicparty	Positive
4	illegal parking commercial overnight parking	Negative

4. Sentiment Distribution Visualization

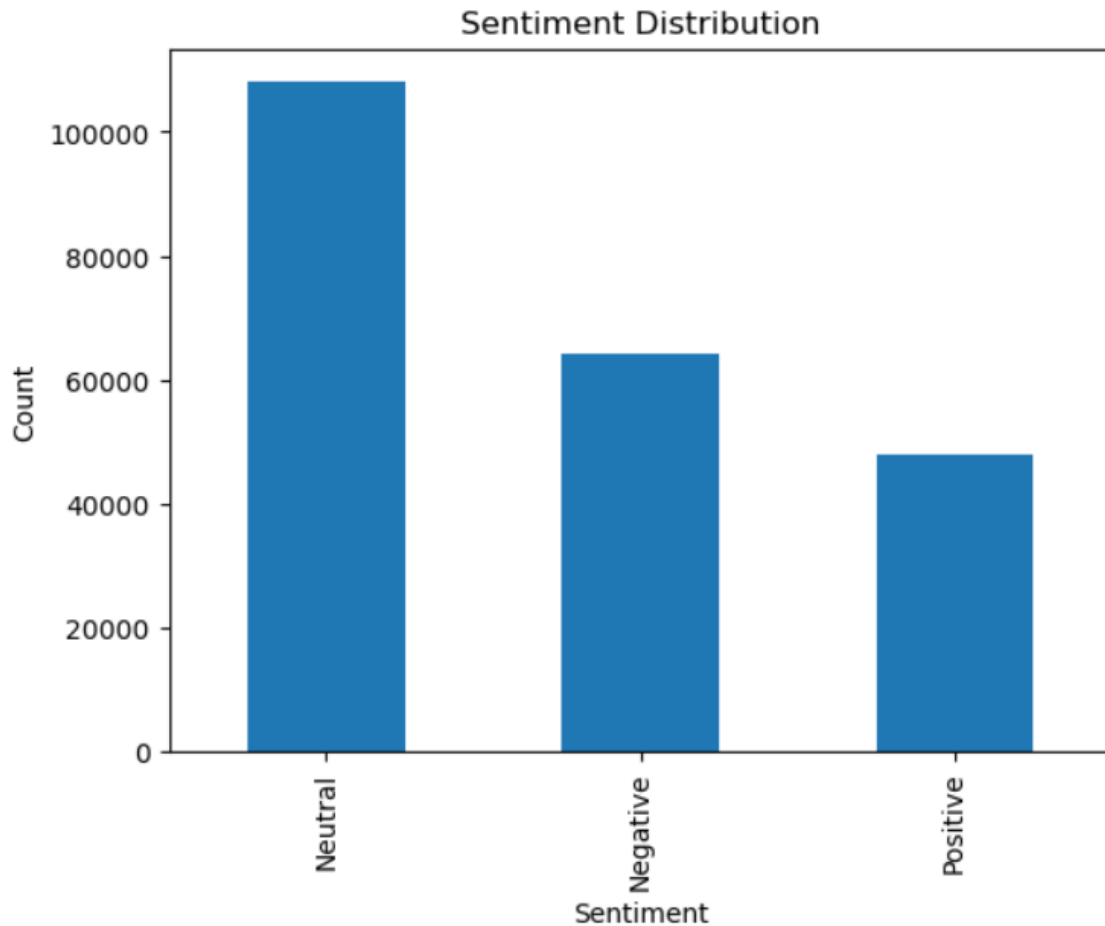
A bar chart was created to visualize the distribution of sentiments across all complaints. This helps in understanding the overall emotional trend of citizen feedback.

```
*[34]: import matplotlib.pyplot as plt

df['sentiment'].value_counts().plot(kind='bar')

plt.title("Distribution of Complaint Sentiments")
plt.xlabel("Sentiment")
plt.ylabel("Count")

plt.show()
```



5. Priority Scoring System

To improve complaint handling, a priority level was assigned based on sentiment. Complaints with negative sentiment were considered more urgent compared to neutral and positive ones.

Priority Mapping:

- Negative → High Priority
- Neutral → Medium Priority
- Positive → Low Priority

```
[39]: def get_priority(sentiment):  
      if sentiment == "Negative":  
          return "High"  
      elif sentiment == "Neutral":  
          return "Medium"  
      else:  
          return "Low"  
  
      df['priority'] = df['sentiment'].apply(get_priority)  
  
[40]: df[['clean_text', 'sentiment', 'priority']].head()  
  
[40]:
```

	clean_text	sentiment	priority
0	noise commercial loud musicparty	Positive	Low
1	noise residential bangingpounding	Neutral	Medium
2	illegal parking blocked hydrant	Negative	High
3	noise residential loud musicparty	Positive	Low
4	illegal parking commercial overnight parking	Negative	High

7. Key Insights

- Most complaints fall under Negative and Critical categories, indicating dissatisfaction among citizens
- Sentiment analysis helps in identifying urgent issues automatically
- Priority scoring enables efficient handling of complaints based on severity
- The system reduces manual effort in sorting and prioritizing complaints

8. Conclusion

In Week 3, sentiment analysis was successfully implemented to classify complaints based on their emotional tone. A priority scoring system was also developed to rank complaints according to urgency. This approach enhances the efficiency of grievance management systems by enabling faster identification and resolution of critical issues.

Future improvements may include the use of advanced transformer-based models such as BERT for more accurate sentiment classification.